Czym są Inteligentne wskaźniki oraz funkcje lambda?

Inteligentne wskaźniki oraz funkcje lambda to kluczowe mechanizmy nowoczesnego C++ (C++11 i nowszych), które ułatwiają zarządzanie pamięcią i pisanie zwięzłego, czystego kodu.

1. Inteligentne wskaźniki (Smart Pointers)

Inteligentny wskaźnik to obiektowa „otoczka" (klasa wyższego poziomu) wokół zwykłego wskaźnika (`T*`), która automatycznie zarządza czasem życia dynamicznie alokowanej pamięci na stercie (ang. *heap*).

Działa na zasadzie **RAII** (*Resource Acquisition Is Initialization*): obiekt alokuje pamięć przy tworzeniu, a w swoim destruktorze automatycznie zwalnia ją (`delete`), gdy wychodzi z zakresu widzialności (np. z klamry funkcji). Dzięki temu nie trzeba ręcznie wywoływać `delete` ani pamiętać o zwalnianiu pamięci w przypadku zgłoszenia wyjątku.

W bibliotece standardowej `<memory>` znajdują się trzy główne typy:

A. `std::unique_ptr` **(Wyłączna własność)**

- **Zasada działania:** Tylko jeden wskaźnik może być właścicielem danej pamięci w danej chwili.
- **Właściwości:** Nie można go kopiować (kopiowanie jest zablokowane na poziomie kompilacji), ale można przenosić własność za pomocą `std::move`.
- **Tworzenie:** Zaleca się tworzenie go przez `std::make_unique<T>(args)`.
- **Zastosowanie w projekcie:** W pliku `PLAN.html` używamy go do zarządzenia obiektami strategii AI w funkcji `makeStrategy` oraz w klasie `Game`. Dzięki temu gra automatycznie usuwa obiekt strategii z pamięci po zakończeniu partii, bez używania `delete`.

```
C++

// Przykładowe użycie w C++17:
std::unique_ptr<Strategy> strategy = std::make_unique<GreedyStrategy>();
// Pamięć zostanie zwolniona automatycznie, gdy zmienna 'strategy' wyjdzie z zakresu!
```

B. `std::shared_ptr` **(Współdzielona własność)**

- **Zasada działania:** Wiele wskaźników może odwoływać się do tego samego obiektu. Wewnątrz prowadzi licznik odwołań (ang. *reference count*).
- **Właściwości:** Można go kopiować. Pamięć zostanie zwolniona dopiero wtedy, gdy ostatni `shared_ptr` wskazujący na ten obiekt zostanie zniszczony.

C. `std::weak_ptr` **(Słabe odniesienie)**

- **Zasada działania:** Wskaźnik, który podgląda obiekt zarządzany przez `std::shared_ptr`, ale nie zwiększa jego licznika odwołań.
- **Właściwości:** Używany głównie do zapobiegania wyciekom pamięci wynikającym z cyklicznych zależności (np. gdy dwa obiekty wskazują na siebie nawzajem przez `shared_ptr`).

2. Funkcje Lambda (Wyrażenia Lambda)

Funkcja lambda to anonimowa funkcja zdefiniowana wprost w miejscu jej użycia w kodzie. Jest powszechnie stosowana jako argument dla algorytmów z biblioteki STL (np. `std::sort`, `std::find_if`, `std::for_each`).

Ogólna budowa składniowa:

```
C++

[ capture ] ( parameters ) -> return_type { body }
```

1. `[ capture ]` **(Klauzula przechwytywania):** Określa, jakie zmienne z otoczenia (z funkcji zewnętrznej) lambda może odczytać lub zmodyfikować:
2.
    - `[]` – brak przechwytywania zmiennych zewnętrznych.
    - `[=]` – przechwytuje wszystkie zmienne Lokalne przez wartość (tworzy kopie).
    - `[&]` – przechwytuje wszystkie zmienne Lokalne przez referencję.
    - `[this]` – przechwytuje wskaźnik na bieżący obiekt klasy, dając dostęp do jej pól i metod.
3. `( parameters )`**:** Lista argumentów przekazywanych do lambdy (tak jak w zwykłej funkcji).
4. `-> return_type`**:** Opcjonalny typ zwracany (zazwyczaj kompilator sam go dedukuje).
5. `{ body }`**:** Kod funkcji.

Zastosowanie w projekcie:

W pliku `Board.cpp` lambda została użyta wewnątrz metody `legalMoves()` do posortowania dostępnych kolumn tak, aby AI najpierw sprawdzało kolumny położone najbliżej środka planszy:

```
C++
std::stable_sort(moves.begin(), moves.end(), [this](int a, int b) {
    return std::abs(2 * a - columns() + 1) < std::abs(2 * b - columns() + 1);
});
```

- `[this]` pozwala lambdzie wywołać prywatną metodę `columns()` obiektu klasy `Board`.
- `(int a, int b)` to dwie porównywane kolumny przekazywane przez `std::stable_sort`.
- **Ciało funkcji** zwraca `true`, jeśli kolumna `a` jest bliżej środka niż kolumna `b`.